CIS5200: Machine Learning Spring 2026

21: Sampling

Date: April 9, 2026

Acknowledgements. These notes are based on Lecture 4 of the Probabilistic Inference and Learning course from University of Tübingen.

Throughout this course, we have defined many probability distributions — from logistic regression to Gaussian processes to mixture models. But defining a distribution and actually *using* it are very different things. Suppose you have a probabilistic model $p(x)$ and want to do any of the following: generate synthetic data that looks realistic, compute an expectation $\mathbb{E}_p[f(x)]$ that has no closed-form solution, or quantify how uncertain a prediction is. All of these tasks reduce to the same fundamental problem: **how do you draw samples from a distribution $p(x)$?**

For simple distributions like Gaussians or uniforms, this is straightforward — your computer already knows how. But for the complex, high-dimensional distributions that arise in practice (e.g., the distribution over all realistic images of faces), sampling becomes genuinely hard. This lecture develops the key tools for tackling this problem: **Monte Carlo methods** for using samples to estimate integrals, and sampling algorithms — rejection sampling, importance sampling, and Markov Chain Monte Carlo — for drawing from distributions we cannot sample from directly. These techniques will be essential building blocks when we turn to generative models (VAEs, diffusion models) in the coming lectures.

1 Probabilistic models

Let's ground this with concrete examples. Recall the probabilistic models we have encountered so far:

1. In logistic regression, we defined $p(y|x) = \sigma(w^\top x)$
2. In Gaussian processes, we defined
$$p\left(\begin{bmatrix} y_x \\ y_z \end{bmatrix} \middle| \begin{bmatrix} x \\ z \end{bmatrix}\right) = \mathcal{N}\left(0, \begin{bmatrix} K_{xx} & K_{xz} \\ K_{xz}^\top & K_{zz} \end{bmatrix}\right)$$
3. In Gaussian mixture models, we defined $p(x) = \sum_k \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$

In each case, we defined a distribution p but mostly used it to learn parameters or make point predictions. What if we want to go further — simulate new examples from the distribution, compute an expectation that lacks a closed form, or normalize an unnormalized density $\hat{p}(x)$ by computing $Z = \int \hat{p}(x)dx$? All of these require the ability to **sample** from $p(x)$ and to use those samples to approximate integrals. This is exactly the idea behind Monte Carlo methods.

1.1 Monte Carlo Methods

Monte Carlo methods estimate an integral with samples. In the general form, it is the idea that for a distribution $p(x)$ and B samples $x_1, \ldots, x_B \sim p(x)$, we can use the following approximation:

$$\mathop{\mathbb{E}}_{x \sim p(x)}[f(x)] = \int_x f(x)p(x)dx \approx \frac{1}{B}\sum_{i=1}^{B} f(x_i)$$

This is nothing more than the average over observations from the distribution. This is a very general strategy that does not require the design of an elaborate integration algorithm.

Furthermore, the error is independent of the dimensionality of the input. However, this requires being able to sample $x_i \sim p(x)$. In these notes, we'll discuss ways in which we can sample from a distribution.

Unbiasedness. Monte Carlo sampling works on *every* integrable function. Formally, we mean that the Monte Carlo estimate of the integral is an unbiased estimate of the true integral. If we say

$$\phi = \int_x f(x)p(x)dx = \mathop{\mathbb{E}}_{p}[f(x)]$$

and consider the Monte Carlo estimate

$$\hat{\phi} = \frac{1}{B}\sum_{i=1}^{B} f(x_i)$$

where $x_i \sim p(x)$, then

$$\mathop{\mathbb{E}}_{p}[\hat{\phi}] = \int_x \frac{1}{B}\sum_{i=1}^{B} f(x_i)p(x_i)dx_i = \frac{1}{B}\sum_{i=1}^{B}\int_x f(x_i)p(x_i)dx_i = \frac{1}{B}\sum_{i=1}^{B}\mathop{\mathbb{E}}_{p}[f(x_i)] = \phi$$

which is an unbiased estimator. The only requirement is that the integral must exist; beyond that, this works even for discontinuous functions.

Convergence. Monte Carlo methods are quite general, but this comes at a cost—their convergence can be quite slow. Why is this the case? Let's look at the expected error of the estimator:

$$\mathrm{Var}(\hat{\phi}) = \mathbb{E}[(\hat{\phi} - \phi)^2]$$

Since the samples $x_1, \ldots, x_B$ are drawn independently from $p(x)$, the random variables $f(x_1), \ldots, f(x_B)$ are i.i.d. The variance of a sum of independent random variables equals the sum of the variances, so

$$\begin{aligned}
\mathrm{Var}(\hat{\phi}) &= \mathrm{Var}\left(\frac{1}{B}\sum_{i=1}^{B} f(x_i)\right) && \text{(definition of } \hat{\phi}\text{)} \\
&= \frac{1}{B^2}\sum_{i=1}^{B}\mathrm{Var}(f(x_i)) && \text{(independence: Var}\left(\textstyle\sum\right) = \textstyle\sum \mathrm{Var}\text{)} \\
&= \frac{1}{B^2}\cdot B \cdot \mathrm{Var}(f(x)) && \text{(identically distributed: each term is the same)} \\
&= \frac{1}{B}\mathrm{Var}(f(x))
\end{aligned}$$

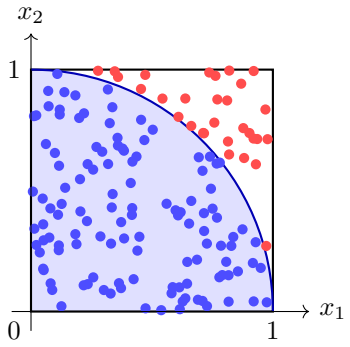

Figure 1: Estimating π by sampling: points drawn uniformly in the unit square are colored by whether they fall inside (blue) or outside (red) the quarter circle. The fraction inside, times 4, estimates π.

which shows that the expected error drops as $O(B^{-1/2})$. Therefore, you should not expect very precise answers with Monte Carlo estimators. To get within an order of magnitude, you need approximately 10 samples. However, to get single precision (within $\epsilon = 10^{-7}$) you need 10^{14} samples. It is wise to only use sampling for rough estimates and consider other options before resorting to sampling.

2 Sampling

As an exercise, consider the number π. Imagine you were trying to estimate the value of π but did not have a calculator. However, we know that π is the area of the unit circle (a circle of radius one). Can we use this to estimate the value of π via sampling? One simple idea here is the following: suppose we know how to draw uniformly random numbers in the interval $[0, 1]$. Then a simple process to estimate π is to

1. Draw $x_1, \ldots, x_B \sim \text{Uniform}([0, 1]^2)$
2. Count the number B' of points satisfying $x_i^\top x_i \leq 1$
3. Return $\frac{4B'}{B}$

Why does this work? A quarter circle has area $\pi/4$, while the unit square has area 1. Therefore, the quarter circle takes up a $\pi/4$ fraction of the unit square. If we sample uniformly over the unit square, we would expect a fraction $\pi/4$ of the points to land within the quarter circle. Scaling up this fraction by 4 gets us the number π.

More formally, this can be seen as estimating the integral of an indicator function

$$\pi = \int_x \mathbf{1}[x \in \text{UnitCircle}]\, dx$$

where $\mathbf{1}[x \in \text{UnitCircle}]$ is 1 if x is in the unit circle and 0 otherwise. To show how simple Monte Carlo methods are as an algorithm, the code for this sampling-based estimator for π is:

```
>>> import torch
>>> B = 100000
>>> r2 = (torch.rand(B,2)**2).sum(axis=1)
>>> pi = (r2<=1).float().mean()*4
>>> pi
tensor(3.1389)
```

We will assume for now that we can sample from a basic distribution, such as the Uniform distribution or the Normal distribution. One statistical way of sampling from more complex distributions is to transform the Uniform distribution to other distributions. For example, if $u \sim \text{Uniform}(0, 1)$ then $x = u^{1/\alpha} \sim \text{Beta}(\alpha, 1)$. However, such transformations require deriving an explicit relationship between Uniform and the target distribution, which can be a hard or impossible problem in general. How do we sample without such an explicit map?

2.1 Rejection sampling

A simple method for sampling, from Georges-Louis Leclerc and Comte de Buffon (1707-1788), is called rejection sampling. The idea is quite similar to the circle example, in that we overapproximate the target distribution and then reject samples that lie outside the target. It uses the following procedure:

1. Let $\tilde{p}(x)$ be any (possibly unnormalized) distribution.
2. Choose any known $q(x)$ such that $cq(x) \geq \tilde{p}(x)$.
3. Draw $s \sim q(x)$ and $u \sim \text{Uniform}[0, cq(s)]$.
4. Reject the sample if $u > \tilde{p}(s)$, otherwise return s as a sample.

An example of the upper bound $cq(x)$ is shown in Figure 2.

Why unnormalized targets are fine. Write $\tilde{p}(x) = Z \cdot p(x)$ where Z is an unknown normalizing constant. The density of accepted samples at x is

$$q(x) \cdot \frac{\tilde{p}(x)}{cq(x)} = \frac{\tilde{p}(x)}{c} \propto p(x)$$

so accepted samples follow the normalized $p(x)$ regardless of whether we know Z. This is key in practice, since many distributions of interest (e.g., Bayesian posteriors) are only known up to a normalizing constant.

Acceptance rate. How efficient is rejection sampling? For a given proposal s, we accept when $u \leq \tilde{p}(s)$ where $u \sim \text{Uniform}[0, cq(s)]$, so ra

$$P(\text{accept} \mid s) = \frac{\tilde{p}(s)}{cq(s)}$$

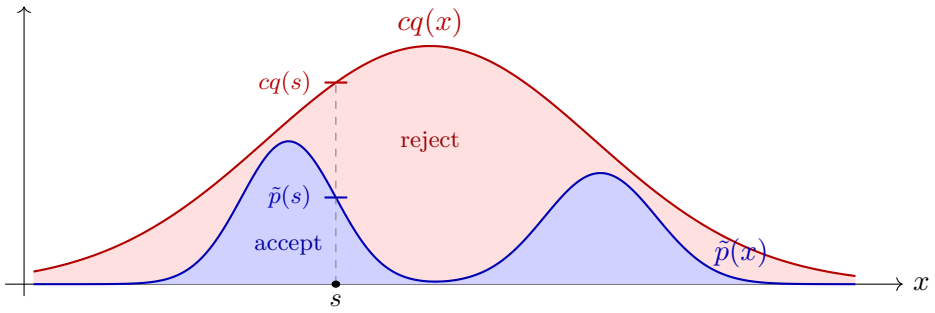

Figure 2: Rejection sampling: the target $\tilde{p}(x)$ (blue, here a bimodal distribution) is bounded above by a scaled proposal $cq(x)$ (red). A sample s is drawn from $q(x)$, then $u \sim \text{Uniform}[0, cq(s)]$ is drawn. If $u \leq \tilde{p}(s)$ the sample is accepted; otherwise it is rejected. The shaded regions show the relative acceptance and rejection areas.

Averaging over all proposals $s \sim q(x)$ gives the overall acceptance probability:

$$P(\text{accept}) = \int \frac{\tilde{p}(s)}{cq(s)} q(s)\, ds = \frac{1}{c} \int \tilde{p}(s)\, ds = \frac{1}{c}$$

where the last equality holds when $\tilde{p}$ is a normalized density. We therefore want c as small as possible. The tightest valid c is $c = \max_x p(x)/q(x)$, the worst-case ratio of the two densities. In general, the tighter the upper bound, the more efficient the scheme. If the bound is too loose, many samples get rejected and a lot of time is spent re-sampling.

Curse of dimensionality. To see how this scales with dimension, consider a simple case: both p and q are zero-mean Gaussians in $\mathbb{R}^D$, with $p = \mathcal{N}(0, \sigma_p^2 I)$ and $q = \mathcal{N}(0, \sigma_q^2 I)$ where $\sigma_q > \sigma_p$ (so q is wider). We can write the ratio explicitly:

$$\frac{p(x)}{q(x)} = \left(\frac{\sigma_q}{\sigma_p}\right)^D \exp\left(-\frac{\|x\|^2}{2}\left(\frac{1}{\sigma_p^2} - \frac{1}{\sigma_q^2}\right)\right)$$

Since $\sigma_p < \sigma_q$, the term $\frac{1}{\sigma_p^2} - \frac{1}{\sigma_q^2} > 0$, so the exponent is always non-positive and equals zero only at $x = 0$. In other words, p is more concentrated at the origin than q, so p decays faster as $\|x\|$ grows, and the ratio is maximized at $x = 0$:

$$c = \frac{p(0)}{q(0)} = \left(\frac{\sigma_q}{\sigma_p}\right)^D$$

The acceptance ratio $1/c = (\sigma_p/\sigma_q)^D$ therefore decays *exponentially* in D. Even for the modest ratio $\sigma_q/\sigma_p = 2$, in $D = 100$ dimensions the acceptance rate is $2^{-100} \approx 10^{-30}$. We would need roughly 10^{30} proposals to get a single accepted sample. This is a fundamental difficulty: rejection sampling becomes impractical in high dimensions, even when p and q are both simple Gaussians.

2.2 Importance sampling

Instead of throwing away samples, another strategy is to use all samples but weight them by importance. Specifically, we can rewrite the integral as

$$\phi = \int f(x)p(x)dx = \int f(x)\frac{p(x)}{q(x)}q(x)dx \approx \frac{1}{B}\sum_{i=1}^{B} f(x_i)\frac{p(x_i)}{q(x_i)}$$

where $x_i \sim q(x)$. In essence, we are simply using Monte Carlo estimation on a new function where samples are drawn from a known distribution $q(x)$. The ratio $\frac{p(x)}{q(x)}$ is known as the importance weight of x.

This sounds great: we have made no assumptions on p or q (other than q must have support that subsumes the support of p to avoid division by zero), and we can use all samples from q! A simple choice for q is to then use the Normal distribution for everything, since it has support everywhere and we can sample from it.

The variance problem. Unfortunately it turns out that this is too good to be true. Recall that the error of our Monte Carlo estimator depends on $\text{Var}(f(x))$, which we had treated as an ignorable constant. Since we are now estimating $f(x)\frac{p(x)}{q(x)}$, the error of our estimator is now dependent on $\text{Var}\left(f(x)\frac{p(x)}{q(x)}\right)$. This variance can no longer be ignored, because it can be arbitrarily large when $q \ll p$! Therefore, even though none of the samples from q are discarded, the total number of samples needed to reduce the error can wind up even larger than for rejection sampling.

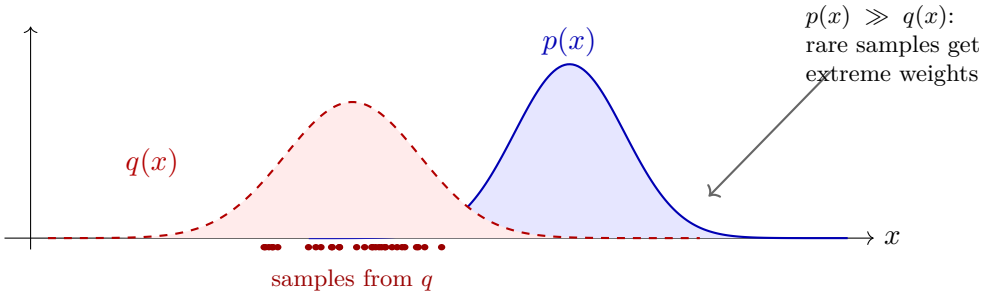

Figure 3: The danger of importance sampling with a poorly chosen proposal. Most samples from $q(x)$ (dashed, red) land where $p(x)$ (blue) has little mass. The few samples that do fall in p's high-density region carry enormous importance weights $p(x)/q(x)$, causing high variance.

2.3 Markov Chain Monte Carlo

Both rejection sampling and importance sampling try to draw *independent* samples from p using a single global proposal q. As we saw, this fails in high dimensions because no single q can cover p well everywhere. Markov Chain Monte Carlo (MCMC) takes a fundamentally different approach: instead of independent samples, it builds a *sequence* $x^{(1)}, x^{(2)}, \ldots, x^{(T)}$ where each sample depends on the previous one. The sequence is constructed so that its distribution converges to the target $p(x)$, allowing us to treat later samples as approximately drawn from p.

The key idea is to use **local proposals**: given the current state $x^{(t)}$, propose a small perturbation $x' \sim q(x' \mid x^{(t)})$ rather than sampling from a fixed global distribution. Because the proposal is local, it can be well-calibrated to p in the neighborhood of the current point—avoiding the curse of dimensionality. The most famous MCMC method, **Metropolis–Hastings** (Metropolis et al., 1953; Hastings, 1970), repeatedly proposes such local moves and accepts or rejects them based on the ratio $p(x')/p(x^{(t)})$. Like rejection sampling, normalizing constants cancel in this ratio, so MCMC also works with unnormalized targets.

MCMC is a large and rich topic—for details on specific algorithms (Metropolis–Hastings, Gibbs sampling, Hamiltonian Monte Carlo) and convergence diagnostics, see Lecture 5 from the University of Tübingen class on Probabilistic Machine Learning.